第 5 章 队列和 queue

1. 队列的概念

（可以用 ppt 演示一下队列的相关概念）

队列也是一种**访问受限**的线性表，它只允许在表的一端进行插入操作，在另一端进行删除操作。

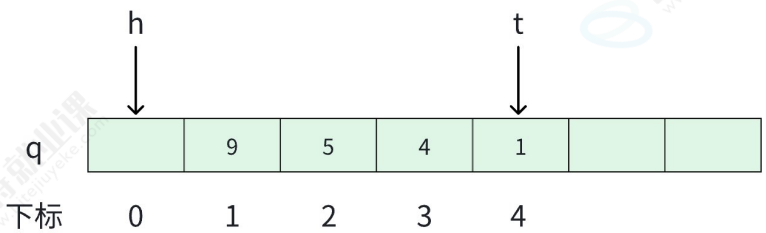
- 允许插入的一端称为**队尾**，允许删除的一端称为**队头**。
- 先进入队列的元素会先出队，故队列具有**先进先出(First In First Out)**的特性。

2. 队列的模拟实现

（比较简单，就不用 ppt 展示了，用课件上的图足矣）

2.1 创建

> - 一个足够大的数组充当队列；
> - 一个变量 `h` 标记队头元素的**前一个位置**；
> - 一个变量 `t` 标记队尾元素的位置。
>
> 两个变量 `(h, t]` 是一种**左开右闭**的形式，这样设定纯属个人喜好，因为后续的代码写着比较舒服。
>
> 当然，也可以 `h` 标记队头元素的位置。只要能控制住代码不出现 `bug`，想怎么实现就怎么实现。

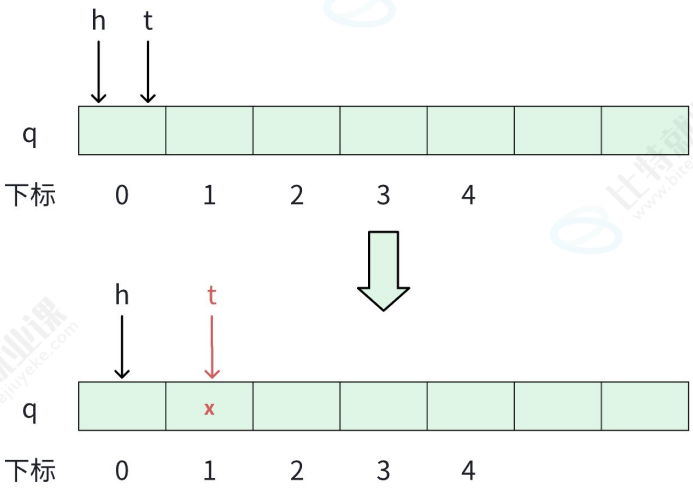



```
1   const int N = 1e6 + 10;
2
3   int h, t; // 队头指针，队尾指针
4   int q[N]; // 队列
```

2.2 入队

- 将 x 这个元素放入到队列中。

注意，我们这里依旧从下标为 1 的位置开始存储有效元素。



代码实现：

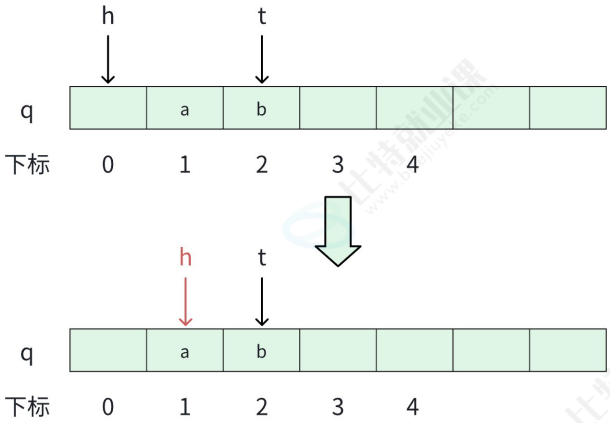
```
// 入队
void push(int x)
{
    q[++t] = x;
}
```

时间复杂度：

显然是 $O(1)$。

2.3 出队

- 删除队头元素。



代码实现：

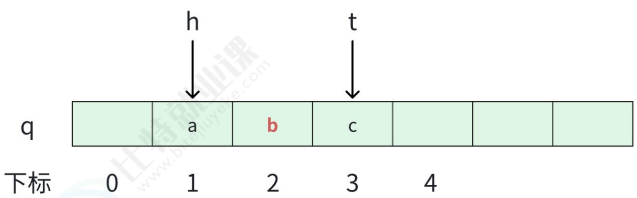
```
// 出队
void pop()
{
    // 有没有 bug?
    h++;
}
```

时间复杂度：

> 显然是 $O(1)$。

2.4 队头

> - 返回队列里面的第一个元素。
>
> 这里要注意，不是 h 所指的位置，而是 h 所指的下一个位置。



代码实现：

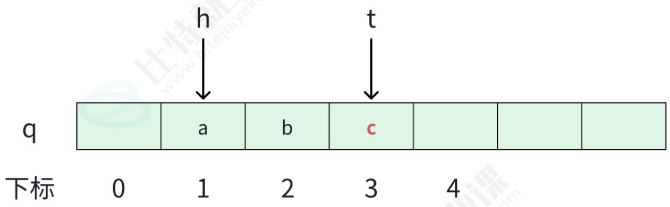
```
// 队头元素
int front()
{
    return q[h + 1];
}
```

时间复杂度：

> 显然是 $O(1)$。

2.5 队尾

- 返回队列里面的最后一个元素



代码实现：

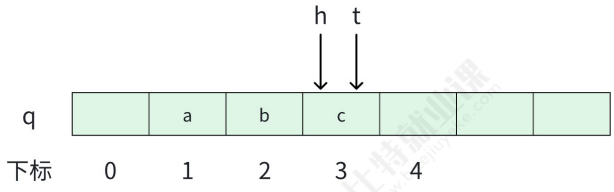
```
// 队尾元素
int back()
{
    return q[t];
}
```

时间复杂度：

> 显然是 $O(1)$。

2.6 判空

- 判断队列是否为空。



代码实现：

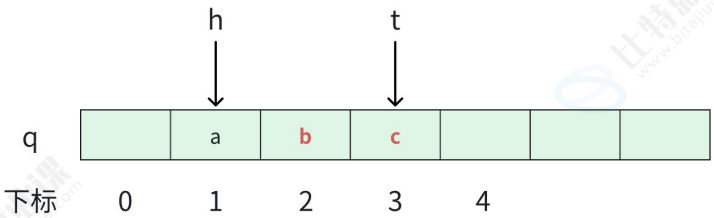
```
// 队列是否为空
bool empty()
{
    return t == h;
}
```

时间复杂度：

> 显然是 $O(1)$。

2.7 元素个数

> - 返回队列里面元素的个数。



代码实现：

```
// 队列的大小
int size()
{
    return t - h;
}
```

时间复杂度：

> 显然是 $O(1)$。

2.8 所有测试代码

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

// 创建
int q[N], h, t;

// 入队
void push(int x)
{
    q[++t] = x;
}

// 出队
void pop()
{
    h++;
}

// 查询队头元素
int front()
{
    return q[h + 1];
}

// 查询队尾元素
int back()
{
    return q[t];
}

// 判断是否为空
bool empty()
{
```

```
37        return h == t;
38    }
39
40    // 有效元素的个数
41    int size()
42    {
43        return t - h;
44    }
45
46    int main()
47    {
48        // 测试
49        for(int i = 1; i <= 10; i++)
50        {
51            push(i);
52        }
53
54        while(size()) // while(!empty())
55        {
56            cout << front() << " " << back() << endl;
57            pop();
58        }
59
60        return 0;
61    }
```

3. queue

> 依旧注意三点：
>
> 1. 如何创建？
> 2. 里面提供了什么函数？
> 3. 每个函数的功能以及时间复杂度。

3.1 创建

> `queue<T> q;`
>
> `T` 可以是任意类型的数据。

3.2 size / empty

> 1. `size`：返回队列里实际元素的个数；

2. `empty`：返回队列是否为空。

时间复杂度：$O(1)$。

3.3 push / pop

1. `push`：入队；
2. `pop`：出队。

时间复杂度：$O(1)$。

3.4 front / back

1. `front`：返回队头元素，但不会删除；
2. `back`：返回队尾元素，但不会删除。

时间复杂度：$O(1)$。

3.5 所有测试代码

```cpp
#include <iostream>
#include <queue>

using namespace std;

typedef pair<int, int> PII;

int main()
{
    // 测试 queue
    queue<PII> q;

    for(int i = 1; i <= 10; i++)
    {
        q.push({i, i * 10});
    }

    while(q.size()) // while(!q.empty())
    {
        auto t = q.front();
        q.pop();

```

```
            cout << t.first << " " << t.second << endl;
        }

        return 0;
    }
```

4. 算法题

4.1 【模板】队列

> **题目来源：** 洛谷
>
> **题目链接：** B3616 【模板】队列
>
> **难度系数：** ★

【题目描述】

> 请你实现一个队列（queue），支持如下操作：
>
> - `push(x)`：向队列中加入一个数 x。
> - `pop()`：将队首弹出。如果此时队列为空，则不进行弹出操作，并输出 `ERR_CANNOT_POP`。
> - `query()`：输出队首元素。如果此时队列为空，则输出 `ERR_CANNOT_QUERY`。
> - `size()`：输出此时队列内元素个数。

【输入描述】

> 第一行，一个整数 n，表示操作的次数。
>
> 接下来 n 行，每行表示一个操作。格式如下：
>
> - `1 x`，表示将元素 `x` 加入队列。
> - `2`，表示将队首弹出队列。
> - `3`，表示查询队首。
> - `4`，表示查询队列内元素个数。

【输出描述】

> 输出若干行，对于每个操作，按「题目描述」输出结果。
>
> 每条输出之间应当用空行隔开。

【示例一】

> 输入：
>
> ```
> 13
> ```

```
1 2
3
4
1 233
3
2
3
2
4
3
2
1 144
3
```

输出：

```
2
1
2
233
0
ERR_CANNOT_QUERY
ERR_CANNOT_POP
144
```

【解法】

模拟一个队列。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e4 + 10;
```

```cpp

int q[N], h, t;

int main()
{
    int n; cin >> n;
    while(n--)
    {
        int op, x;
        cin >> op;
        if(op == 1) // 进队
        {
            cin >> x;
            q[t++] = x;
        }
        else if(op == 2) // 出队
        {
            if(h == t) cout << "ERR_CANNOT_POP" << endl;
            else h++;
        }
        else if(op == 3) // 队头元素
        {
            if(h == t) cout << "ERR_CANNOT_QUERY" << endl;
            else cout << q[h] << endl;
        }
        else // 元素个数
        {
            cout << t - h << endl;
        }
    }

    return 0;
}
```

4.2 [NOIP2010 提高组] 机器翻译

> **题目来源：** 洛谷
>
> **题目链接：** P1540 [NOIP2010 提高组] 机器翻译
>
> **难度系数：** ★

【题目描述】

> 小晨的电脑上安装了一个机器翻译软件，他经常用这个软件来翻译英语文章。

这个翻译软件的原理很简单，它只是从头到尾，依次将每个英文单词用对应的中文含义来替换。对于每个英文单词，软件会先在内存中查找这个单词的中文含义，如果内存中有，软件就会用它进行翻译；如果内存中没有，软件就会在外存中的词典内查找，查出单词的中文含义然后翻译，并将这个单词和译义放入内存，以备后续的查找和翻译。

假设内存中有 M 个单元，每单元能存放一个单词和译义。每当软件将一个新单词存入内存前，如果当前内存中已存入的单词数不超过 $M-1$，软件会将新单词存入一个未使用的内存单元；若内存中已存入 M 个单词，软件会清空最早进入内存的那个单词，腾出单元来，存放新单词。

假设一篇英语文章的长度为 N 个单词。给定这篇待译文章，翻译软件需要去外存查找多少次词典？假设在翻译开始前，内存中没有任何单词。

【输入描述】

共 2 行。每行中两个数之间用一个空格隔开。

第一行为两个正整数 M, N，代表内存容量和文章的长度。

第二行为 N 个非负整数，按照文章的顺序，每个数（大小不超过 1000）代表一个英文单词。文章中两个单词是同一个单词，当且仅当它们对应的非负整数相同。

对于 100% 的数据有 $1 \leq M \leq 100$，$1 \leq N \leq 1000$。

【输出描述】

一个整数，为软件需要查词典的次数。

【示例一】

输入：

```
3 7
1 2 1 5 4 4 1
```

输出：

```
5
```

【解法】

创建一个队列，模拟整个流程。

同时创建一个 $bool$ 类型的数组，快速判断元素是否在队列中。

【参考代码】

```
#include <iostream>
#include <queue>

```

```cpp
using namespace std;

const int N = 1010;

int m, n;
queue<int> q; // 充当内存
bool st[N]; // st[i] 表示 i 这个元素是否在队列里面

int main()
{
    cin >> m >> n;

    int cnt = 0;
    for(int i = 1; i <= n; i++)
    {
        int x; cin >> x;
        if(st[x]) continue;

        cnt++;
        q.push(x);
        st[x] = true;

        if(q.size() > m)
        {
            st[q.front()] = false;
            q.pop();
        }
    }

    cout << cnt << endl;

    return 0;
}
```

4.3 [NOIP2016 普及组] 海港

> **题目来源：** 洛谷
>
> **题目链接：** P2058 [NOIP2016 普及组] 海港
>
> **难度系数：** ★★

【题目描述】

> 小 K 是一个海港的海关工作人员，每天都有许多船只到达海港，船上通常有很多来自不同国家的乘客。

小 K 对这些到达海港的船只非常感兴趣，他按照时间记录下了到达海港的每一艘船只情况；对于第 i 艘到达的船，他记录了这艘船到达的时间 t_i (单位：秒)，船上的乘客数 k_i ，以及每名乘客的国籍 $x_{i,1}, x_{i,2}, \ldots, x_{i,k}$ 。

小 K 统计了 n 艘船的信息，希望你帮忙计算出以每一艘船到达时间为止的 24 小时（24 小时 $= 86400$ 秒）内所有乘船到达的乘客来自多少个不同的国家。

形式化地讲，你需要计算 n 条信息。对于输出的第 i 条信息，你需要统计满足 $t_i - 86400 < t_p \leq t_i$ 的船只 p ，在所有的 $x_{p,j}$ 中，总共有多少个不同的数。

【输入描述】

第一行输入一个正整数 n ，表示小 K 统计了 n 艘船的信息。

接下来 n 行，每行描述一艘船的信息：前两个整数 t_i 和 k_i 分别表示这艘船到达海港的时间和船上的乘客数量，接下来 个整数 $x_{i,j}$ 表示船上乘客的国籍。

保证输入的 t_i 是递增的，单位是秒；表示从小 K 第一次上班开始计时，这艘船在第 t_i 秒到达海港。

保证 $1 \leq n \leq 10^5$ ， $\sum k_i \leq 3 \times 10^5$ ， $1 \leq x_{i,j} \leq 10^5$ ， $1 \leq t_{i-1} \leq t_i \leq 10^9$ 。

其中 $\sum k_i$ 表示所有的 k_i 的和。

【输出描述】

输出 n 行，第 i 行输出一个整数表示第 i 艘船到达后的统计信息。

【示例一】

输入：

```
3
1 4 4 1 2 2
2 2 2 3
10 1 3
```

输出：

```
3
4
4
```

【示例二】

输入：

```
4
1 4 1 2 2 3
3 2 2 3
```

> 86401 2 3 4
>
> 86402 1 5
>
> 输出：
>
> 3
>
> 3
>
> 3
>
> 4

【解法】

先来的乘客，在时间逐渐增大的时候会先出列。因此，可以用队列模拟整个过程。

- 队列里面存每一个乘客，需要存储乘客的编号还有进队的时间；
- 进队的时候，把队列里面时间差大于等于 24 小时的全出队，然后统计队列里面国家的个数；
- 需要额外创建一个 mp 数组来统计队列里面各个国家的人数，方便在进出队的时候，统计国家的个数。

【参考代码】

```cpp
#include <iostream>
#include <queue>

using namespace std;

typedef pair<int, int> PII;
const int N = 1e5 + 10;

int n;
queue<PII> q; // 统计的是 <时间，国家>
int cnt[N]; // cnt[i] 表示 i 国家一共有多少人
int kinds; // 统计国家的种类

int main()
{
    cin >> n;
    while(n--)
    {
        int t, k; cin >> t >> k;
        for(int i = 1; i <= k; i++)
        {
            int x; cin >> x;
            // <t, x>
```

```
24              q.push({t, x});
25              if(cnt[x]++ == 0) kinds++;
26          }
27
28          // 让队列合法
29          while(q.size() && q.back().first - q.front().first >= 86400)
30          {
31              PII tmp = q.front(); q.pop();
32              int x = tmp.second;
33              if(cnt[x]-- == 1) kinds--;
34          }
35
36          cout << kinds << endl;
37      }
38
39      return 0;
40  }
```

5. 双端队列

（只用在 ppt 中演示一下双端队列的定义，模拟实现不做要求）

双端队列也是一种特殊线性结构，其允许两端都可以进行数据元素入队和出队操作。



将队列的两端分别称为前端和后端，两端都可以进行数据入队和出队。

6. deque

6.1 创建

> `deque<T> q;`
>
> T 可以是任意类型的数据。

6.2 size / empty

1. `size`：返回队列里实际元素的个数；
2. `empty`：返回队列是否为空。

时间复杂度：$O(1)$。

6.3 push_front / push_back

1. `push_front`：头插；
2. `push_back`：尾插。

时间复杂度：$O(1)$。

6.4 pop_front / pop_back

1. `pop_front`：头删；
2. `pop_back`：尾删。

时间复杂度：$O(1)$。

6.5 front / back

1. `front`：返回队头元素，但不会删除；
2. `back`：返回队尾元素，但不会删除。

时间复杂度：$O(1)$。

6.6 clear

`clear`：清空队列。

时间复杂度 $O(N)$。

6.7 所有测试代码

```cpp
#include <iostream>
#include <deque>

using namespace std;

```

```cpp
struct node
{
    int x, y, z;
};

int main()
{
    deque<node> q;

    // 头插
    for(int i = 1; i <= 5; i++)
    {
        q.push_front({i, i * 2, i * 3});
    }

    // 头删
    while(q.size())
    {
        auto t = q.front(); q.pop_front();
        cout << t.x << " " << t.y << " " << t.z << endl;
    }


    // 尾插
    for(int i = 1; i <= 5; i++)
    {
        q.push_back({i, i * 2, i * 3});
    }

    // 尾删
    while(q.size())
    {
        auto t = q.back(); q.pop_back();
        cout << t.x << " " << t.y << " " << t.z << endl;
    }

    return 0;
}
```